

DP Computer Science: A3.1.1 Relational Databases

1. Core Concepts & Learning Objectives

Learning Intentions

By the end of this lesson, students will be able to:

- Describe the key features of a relational database: **tables, primary keys, foreign keys, composite keys, relationships**
 - Explain the benefits of using a relational database: **data retrieval, reduced redundancy, consistency, concurrency control, transaction processing, scalability, security, community support**
 - Discuss the limitations of relational databases: **design complexity, rigid schema, object-relational impedance mismatch, hierarchical data handling, unstructured data, big data scalability**
-

Key Vocabulary

Term	Definition
Relational database	A database that organises data into tables with rows and columns, linked by relationships
Table	A collection of related data organised in rows and columns
Primary key	A unique identifier for each record in a table
Foreign key	A field in one table that refers to the primary key of another table
Composite key	A primary key made up of two or more columns
Relationship	An association between tables based on matching keys

Term	Definition
Normalisation	The process of organising data to reduce redundancy and improve integrity
ACID	Atomicity, Consistency, Isolation, Durability—properties ensuring reliable transactions
Object-relational impedance mismatch	The difficulty of mapping object-oriented programming concepts to relational databases

ATL Skills

Skill	Description
Thinking	Critical thinking (analysing benefits and limitations); knowledge construction (understanding features)
Communication	Articulating explanations; participating in discussions

2. Introduction to Relational Databases

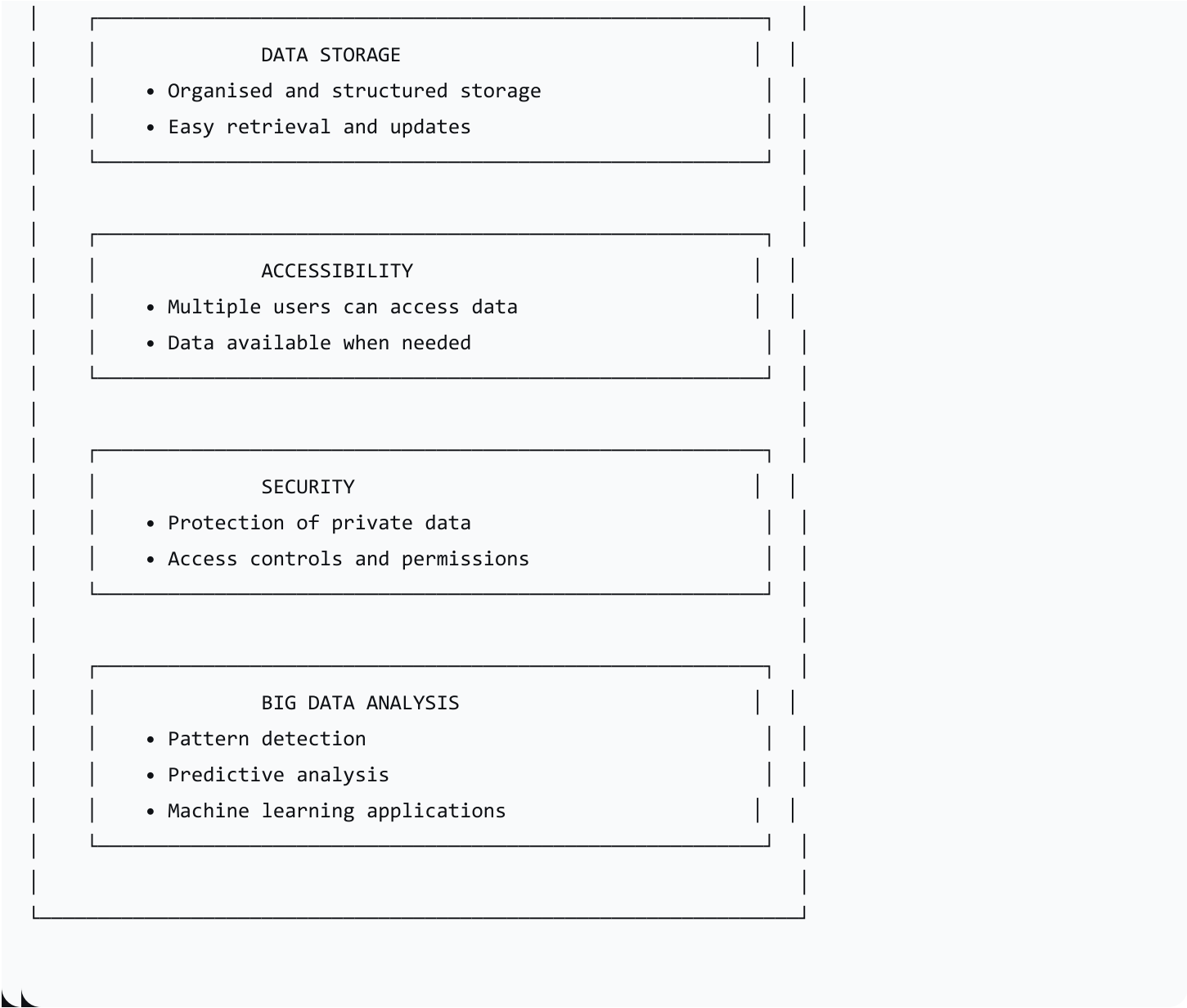
What is a Relational Database?

A relational database is a collection of data organised into tables (relations) with rows and columns. Tables are linked by relationships based on matching keys.

Why Do We Need Databases?

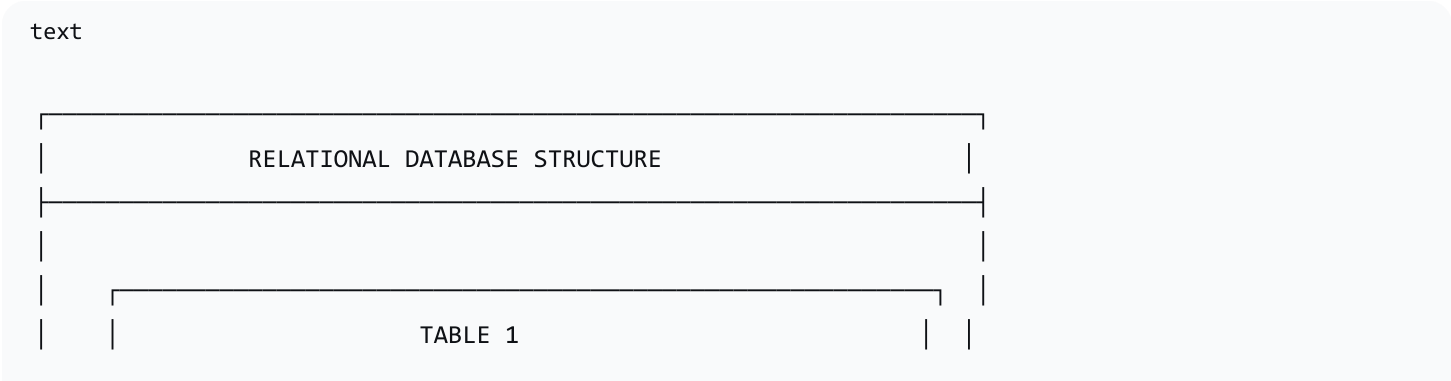
text

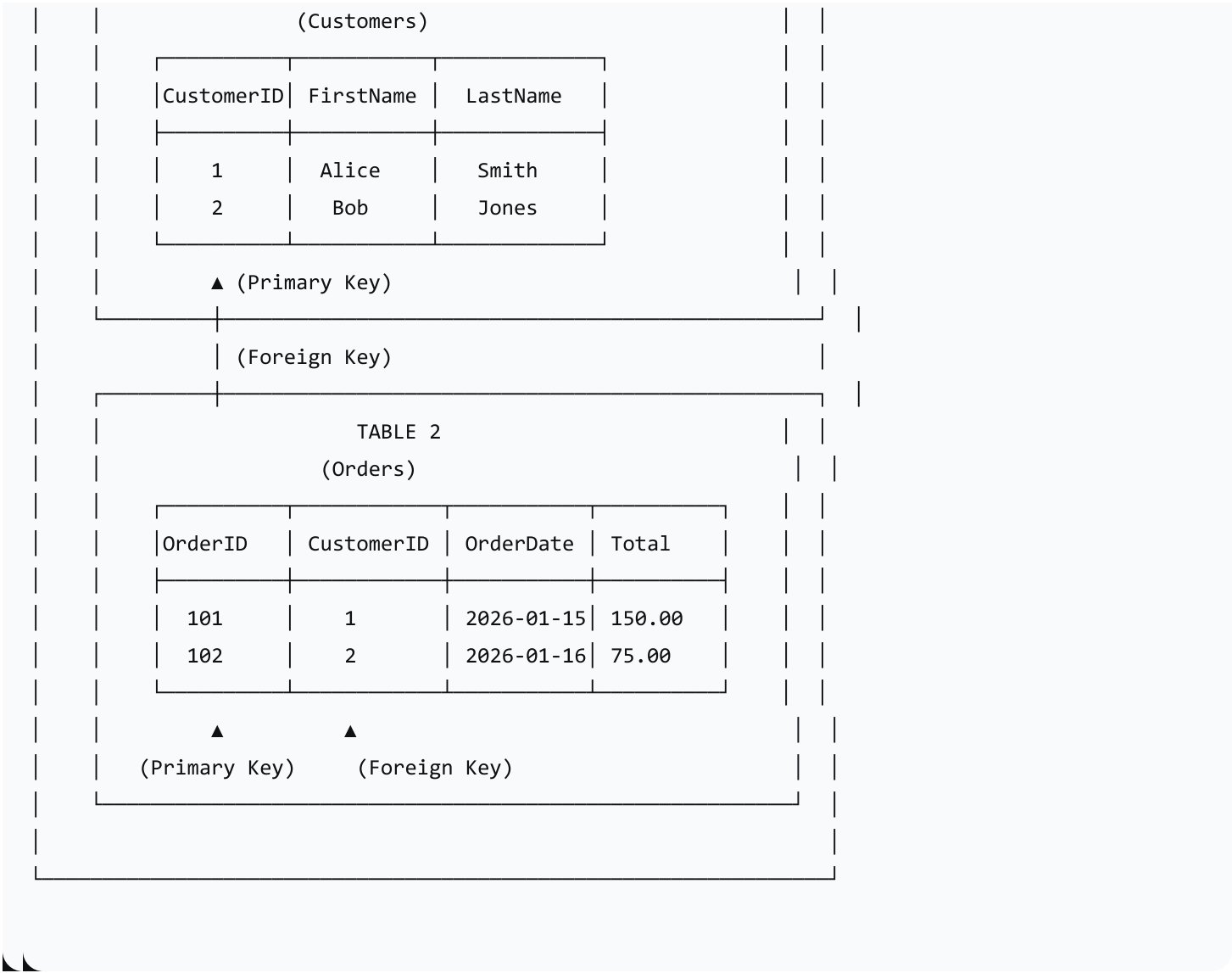
WHY DO WE NEED DATABASES?



3. Key Features of Relational Databases

The Core Structure





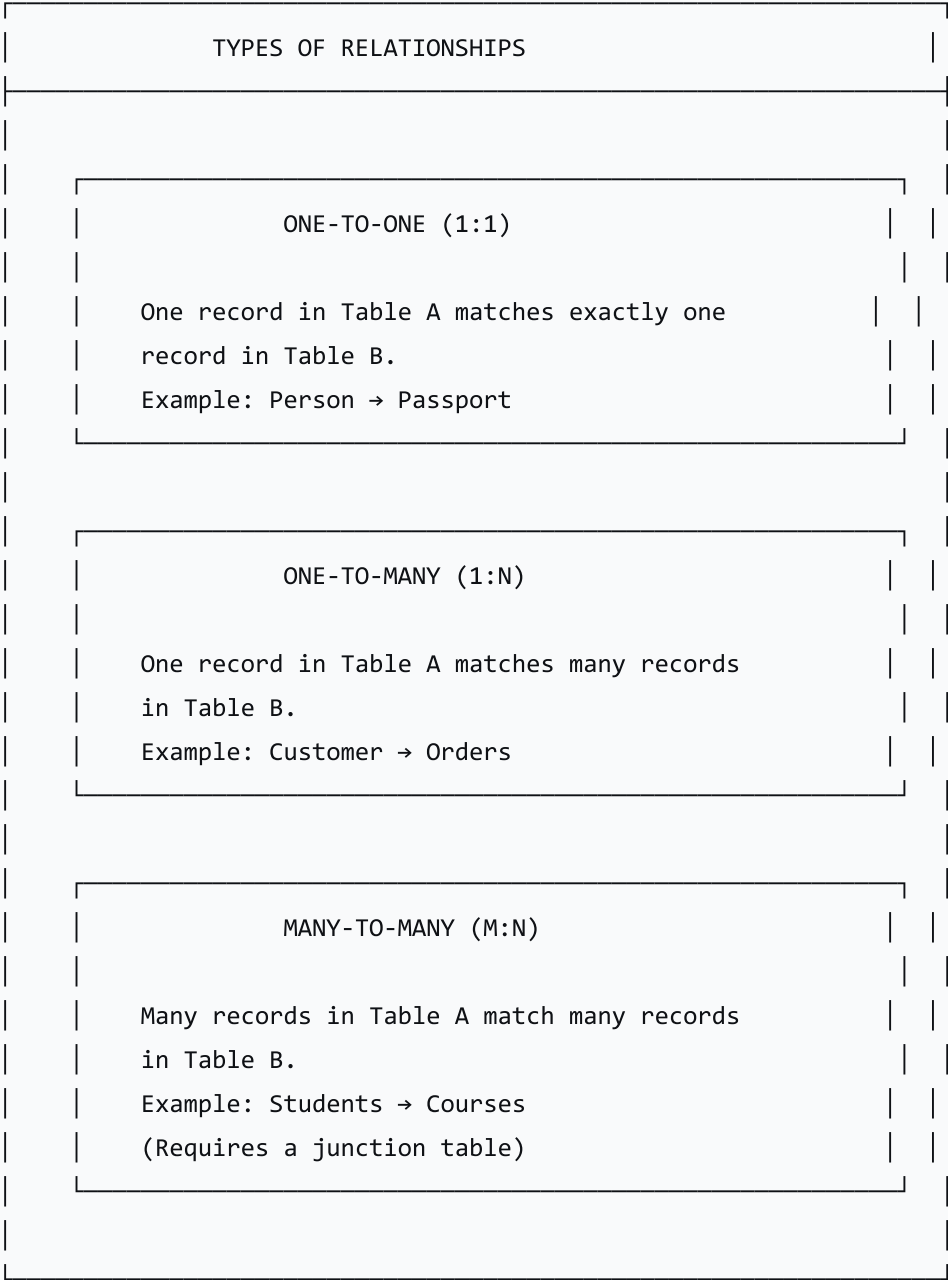
Key Features Explained

Feature	Description	Example
Table	Organises data into rows (records) and columns (fields)	Customers table, Orders table
Primary Key	Uniquely identifies each record in a table	CustomerID in Customers table
Foreign Key	Refers to the primary key of another table	CustomerID in Orders table
Composite Key	A primary key made up of multiple columns	OrderID + ProductID

Feature	Description	Example
Relationship	Links tables together	One-to-many between Customers and Orders

Types of Relationships

text



4. Benefits of Relational Databases

Benefit	Description	Real-World Example
Efficient Data Retrieval	SQL allows fast, flexible querying	<code>SELECT * FROM Customers WHERE City = 'London'</code>
Reduced Redundancy	Normalisation minimises duplicate data	Storing customer details once, not in every order
Data Consistency	Constraints ensure data integrity	Foreign keys prevent orphaned records
Concurrency Control	Multiple users can access data simultaneously	Two users updating the same record without conflict
Transaction Processing	ACID properties ensure reliable transactions	Bank transfer—either both accounts update or neither
Scalability	Can handle increasing amounts of data and users	Adding more servers to handle growth
Security	Access controls protect sensitive data	Only authorised users can view salaries
Community Support	Mature ecosystem, documentation, and tools	Large community for MySQL, PostgreSQL, etc.

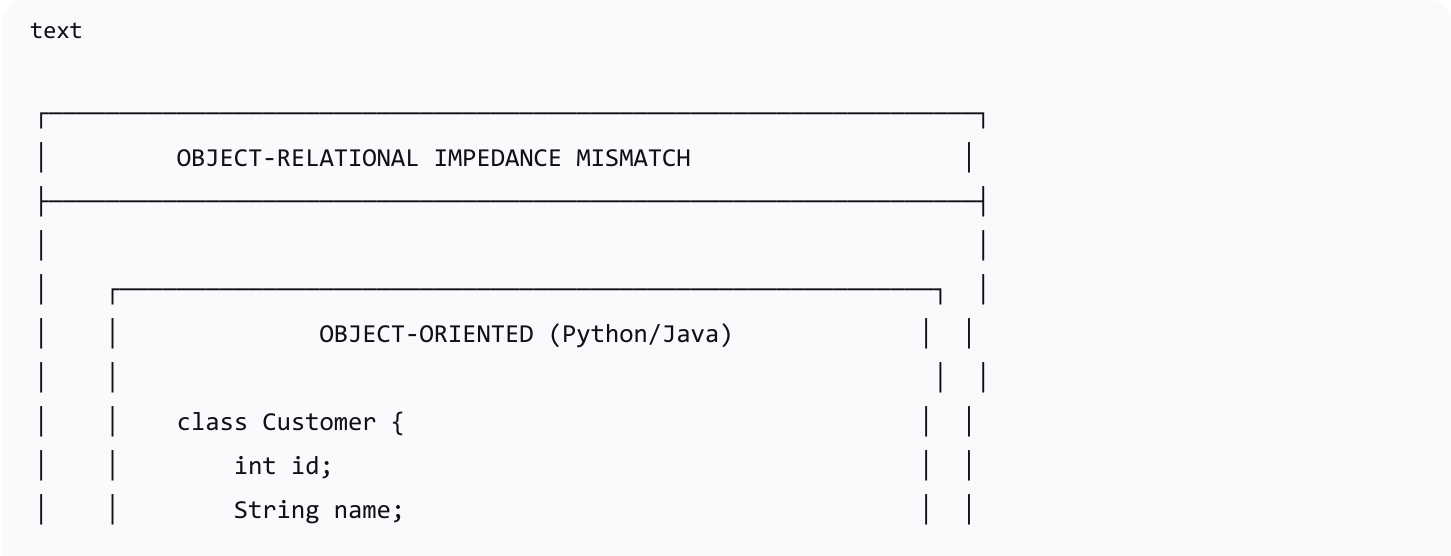
ACID Properties

Property	Description
Atomicity	All or nothing—a transaction is completed fully or not at all
Consistency	Transactions preserve database rules and constraints
Isolation	Transactions do not interfere with each other
Durability	Committed changes are permanent, even after system failure

5. Limitations of Relational Databases

Limitation	Description
Design Complexity	Requires careful planning and design; complex to implement
Rigid Schema	Changing the structure requires migrations; not flexible for evolving data
Object-Relational Impedance Mismatch	Object-oriented programming doesn't map easily to relational tables
Hierarchical Data Handling	Tree-like structures (e.g., organisational charts) are difficult to model
Unstructured Data	Images, videos, and text documents are not easily stored
Big Data Scalability	Horizontal scaling (adding more servers) is challenging
Performance with Complex Queries	Complex JOIN operations can be slow on large datasets

Object-Relational Impedance Mismatch



```
List<Order> orders;  
}
```

- ✓ Objects can contain nested collections
- ✓ Inheritance and polymorphism

Mismatch!

RELATIONAL DATABASE

TABLE Customers

id	name
----	------

1	Alice
---	-------

TABLE Orders

id	customer_id	amount
----	-------------	--------

1	1	150.00
---	---	--------

- ✗ No nested collections or inheritance
- ✗ Data must be flattened into tables

6. Relational vs. Non-Relational Comparison

Feature	Relational (SQL)	Non-Relational (NoSQL)
Data Structure	Tables (rows/columns)	Documents, key-value, graphs
Schema	Fixed/Rigid	Flexible

Feature	Relational (SQL)	Non-Relational (NoSQL)
Scalability	Vertical (scale up)	Horizontal (scale out)
ACID Compliance	Yes	Often eventual consistency
Query Language	SQL	Varies by type
Best For	Structured data, transactions	Unstructured data, big data

7. Activity: Relational vs. Non-Relational Scenarios

Instructions

In groups, analyse each scenario and decide whether a relational database is suitable. Justify your reasoning.

Scenario 1: Social Media Platform

Problem: A social media platform needs to store user profiles, posts, comments, and connections between users.

Analysis	Justification
Recommended	Non-relational (NoSQL)
Why?	Social media data is highly unstructured (posts, comments). Relationships between users are complex and change frequently. NoSQL databases handle this flexibility better and scale horizontally to support millions of users.

Scenario 2: E-Commerce Website

Problem: An e-commerce website needs to store customer information, product details, orders, and inventory.

Analysis	Justification
Recommended	Relational (SQL)
Why?	E-commerce requires strong data consistency and integrity for transactions, orders, and inventory. Relational databases use schemas to define data types and relationships, and SQL enables complex queries for sales analysis.

Scenario 3: Scientific Research Data

Problem: A scientific research project collects large amounts of sensor data, including timestamps and measurements.

Analysis	Justification
Recommended	Non-relational (NoSQL)
Why?	Scientific data often involves large volumes of time-series data. NoSQL databases can handle this efficiently and are designed for scalability and fast data ingestion.

Scenario 4: Content Management System

Problem: A content management system needs to store articles, images, and user comments.

Analysis	Justification
Recommended	Either (or hybrid)

Analysis	Justification
Why?	CMS data includes both structured (user info, metadata) and unstructured (articles, images) content. A relational database works for structured data, while NoSQL offers flexibility for diverse content types.

Scenario 5: Bank Financial System

Problem: A bank needs to store customer accounts, transactions, and financial data.

Analysis	Justification
Recommended	Relational (SQL)
Why?	Banks need strong data integrity and consistency for accurate financial records. Relational databases support ACID properties, which are crucial for reliable financial transactions.

8. Lesson Sequence

Lesson Plan (60 minutes)

Phase	Time	Activity
Introduction	5 min	Review data organisation; introduce databases
Features	10 min	Tables, keys, relationships, normalisation
Benefits	15 min	Data retrieval, redundancy, consistency, concurrency, transactions, scalability, security
Limitations	10 min	Design complexity, rigid schema, impedance mismatch, unstructured data, big data

Phase	Time	Activity
Activity	10 min	Relational vs. Non-Relational scenarios
Wrap-up	5 min	Class discussion; homework assignment

9. Assessment Activities

Assessment Type	Description
Class Participation	Observe engagement and contributions
Activity	Evaluate ability to analyse scenarios and justify choices
Homework	Short essay evaluating the suitability of relational databases

10. Differentiation

Level	Strategy
Support	Provide visual aids, key definitions, and guided examples
Challenge	Research advanced topics: NoSQL vs. SQL in real-world applications

11. Resources

- **Presentation** (Slide Deck)
- **eBook** (A3.1.1 with quiz)
- **Quizlet** (A3.1.1 vocabulary flashcards)
- **Activity:** Relational vs. Non-Relational scenarios

12. Summary: Key Takeaways

Concept	Key Point
Features	Tables, primary keys, foreign keys, relationships
Benefits	Efficient retrieval, reduced redundancy, consistency, transactions, security
Limitations	Rigid schema, impedance mismatch, unstructured data, big data challenges
Best For	Structured data, transactions, consistent data

text

KEY REMINDERS
✓ Relational databases use tables, keys, and relationships ✓ Benefits: consistency, transactions, security ✓ Limitations: rigid schema, impedance mismatch ✓ Not suitable for unstructured data ✓ Not ideal for big data (horizontal scaling)

"Relational databases provide structure, consistency, and reliability—but they are not the right solution for every problem."